



На правах рукописи

Насонов Денис Александрович

**ЭВОЛЮЦИОННЫЕ АЛГОРИТМЫ
РАСПРЕДЕЛЕНИЯ БОЛЬШИХ ДАННЫХ
В ВЫЧИСЛИТЕЛЬНО-ИНТЕНСИВНЫХ ЗАДАЧАХ**

Специальность 05.13.11 – Математическое и программное обеспечение
вычислительных машин, комплексов и
компьютерных сетей

АВТОРЕФЕРАТ

30 НОЯ 2016

диссертации на соискание ученой степени
кандидата технических наук



006660867

Санкт-Петербург – 2016

Работа выполнена в Санкт-Петербургском национальном исследовательском университете информационных технологий, механики и оптики

Научный руководитель: доктор технических наук
Бухановский Александр Валерьевич

Официальные оппоненты: **Гергель Виктор Павлович**
доктор технических наук, профессор, декан факультета ВМК, директор НИИ ПМК, руководитель НОЦ СКТ Нижегородского государственного университета им. Н.И. Лобачевского, Нижний Новгород

Сухорослов Олег Викторович
кандидат технических наук, старший научный сотрудник Лаборатории распределенных вычислительных систем ФГБУ "Институт проблем передачи информации им. А.А. Харкевича Российской академии наук (ИППИ РАН)", Москва

Ведущая организация: Национальный исследовательский центр "Курчатовский институт", Москва

Защита состоится 26 декабря 2016 г. в 18:30 часов на заседании диссертационного совета Д 212.227.06 при Санкт-Петербургском национальном исследовательском университете информационных технологий, механики и оптики по адресу: 197101, Санкт-Петербург, Кронверкский пр., д. 49., ауд. 431.

С диссертацией можно ознакомиться в библиотеке Санкт-Петербургского национального исследовательского университета информационных технологий, механики и оптики по адресу: 197101, Санкт-Петербург, Кронверкский пр., д. 49, а также на сайте http://fppo.ifmo.ru/?page1=16&page2=52&page_d=1&page_d2=135273.

Автореферат разослан « 15 » ноября 2016 года.

Ученый секретарь диссертационного совета
доктор технических наук, профессор



Холодова С. Е.

ОБЩАЯ ХАРАКТЕРИСТИКА РАБОТЫ

Актуальность темы исследования обусловлена интенсивным развитием технологий электронной науки (eScience), предназначенных для поддержки выполнения сложных расчетов с использованием больших объемов данных, размещенных в распределенной вычислительной среде. В России данное направление представлено в работах научных школ А.П. Афанасьева, В.П. Иванникова, В.А. Ильина, В.В. Коренькова и других исследователей. В отличие от классических суперкомпьютерных технологий, особенности работы с большими объемами данных заключаются в том, что эти данные размещаются на распределенных вычислительных узлах не равномерно по объему, а в соответствии со сложной структурой, диктуемой особенностями предметной области. Изменение структуры данных во времени приводит к необходимости их перераспределения для достижения наибольшей эффективности использования вычислительной инфраструктуры. Как следствие, выполнение расчетов над большими данными требует максимизации использования процессорного времени и повышения скорости работы с операциями ввода/вывода за счет использования многоуровневого хранения данных на различных по скорости работы и внутреннему объему устройствах памяти. На сегодняшний день отсутствуют методы и технологии оптимизации работы со сложными структурами больших данных в распределенной среде, которые *совокупно* учитывали бы особенности организации хранения и доступа к данным, а также оптимизацию самих вычислительных процессов, что делает актуальной тему диссертационной работы.

Предметом исследования являются алгоритмы оптимизации размещения больших объемов данных в распределенной вычислительной среде, а также алгоритмы планирования вычислительно-интенсивных задач в распределенных средах на их основе.

Целью исследования является разработка математического, алгоритмического и программного обеспечения для оптимизации работы со сложными структурами больших данных при выполнении расчетов в распределенных вычислительных средах.

Задачи исследования:

- анализ существующих решений для управления большими данными и оптимизацией их размещения, включая алгоритмы планирования, с целью определения требований к объектам исследования и разработки;
- формализация постановки задачи оптимизации размещения данных и планирования на основе параметрических моделей композитных приложений и распределенной вычислительной среды;
- разработка и программная реализация алгоритмов перераспределения больших данных, совмещенных с алгоритмами планирования вычислительно-интенсивных задач в стационарных и нестационарных условиях работы вычислительной среды;

- экспериментальное исследование разработанных алгоритмов для оценки их работоспособности и эффективности, а также сравнения с существующими аналогами.

Методы исследования включают в себя методы теории алгоритмов, дискретной математики, эволюционных вычислений, теории вероятностей и математической статистики, имитационного моделирования, инженерии программного обеспечения и анализа программного обеспечения.

Научная новизна исследования обусловлена тем, что в работе впервые предложено и обосновано семейство алгоритмов распределения сложноструктурированных данных в вычислительной среде, которые позволяют обеспечивать оптимальное размещение данных на вычислительных узлах с использованием ретроспективы эксплуатации системы как в стационарных, так и в нестационарных условиях. На основе указанных алгоритмов разработаны алгоритмы динамического коэволюционного планирования композитных приложений на динамически реконфигурируемых вычислительных ресурсах. По сравнению с аналогами разработанные и программно реализованные алгоритмы позволили повысить эффективность использования вычислительных ресурсов и снизить время обработки больших данных в 1,2–3 раза.

Практическую значимость исследования определяет программная реализация разработанных алгоритмов оптимизации сложных структур больших данных, интегрированная в подсистему хранения данных CLAVIRE/DCStorage и обеспечивающая эффективное функционирование облачной платформы CLAVIRE.

На защиту выносятся:

- семейство алгоритмов перераспределения сложноструктурированных данных на основе ретроспективы эксплуатации вычислительной среды в стационарных и нестационарных условиях;
- семейство алгоритмов динамического коэволюционного планирования композитных приложений на динамически реконфигурируемых вычислительных ресурсах, учитывающих специфику обработки больших сложноструктурированных данных.

Достоверность научных результатов и выводов обусловлена: корректностью формальной постановки задач планирования перераспределения данных и выполняемых на них вычислительно-интенсивных задач; использованием параметрических моделей для оценки экспериментальных данных; результатами экспериментальных исследований на различных конфигурациях распределенной вычислительной среды, сравнений с адаптированными существующими подходами.

Внедрение результатов работы. Результаты работы использованы при выполнении проектов: «Создание высокотехнологичного производства комплексных решений в области предметно-ориентированных облачных вычислений для нужд науки, промышленности, бизнеса и социальной сферы», шифр 2010-218-01-209 по договору от 07 сентября 2010 г. № 13.G25.31.0029, в рамках конкурса субсидий в соответствии с постановлением Правительства Российской Федерации № 218; «Распределенные экстренные вычисления для

поддержки принятия решений в критических ситуациях», дог. № 11.G34.31.0019 от 02.12.2010 г. с дополнительным соглашением № 02 от 01.03.2013 г.; «Информационная технология обеспечения жизненного цикла систем поддержки принятия решений нового поколения для задач персонализированной медицины» № 715788 от 31.03.2015 г.; «Суперкомпьютерное моделирование критических явлений в сложных социальных системах», соглашение № 14-21-00137 от 15.08.2014 г.; «Исследования и разработка быстродействующей кластерной системы хранения и обработки сверхбольших объемов данных», соглашение о предоставлении субсидии от 24 ноября 2014 г. № 14.578.21.0077. В диссертацию включены результаты, полученные при поддержке Министерства образования и науки РФ в рамках договора № 14.Z50.31.0024.

Апробация работы. Основные результаты были представлены на следующих международных научных и научно-практических конференциях: International Conference on Computational Science (Барселона, Испания, 2013); International Conference on Computational Science (Кэрнс, Австралия, 2014); The 14th International Multidisciplinary Scientific GeoConference & EXPO SGEM 2014 (Албена, Болгария, 2014); International Conference on Application of Information and Communication Technologies (Астана, Казахстан, 2014); The 9th International Conference on Soft Computing Models in Industrial and Environmental Applications (Билбао, Испания, 2014); 6th International Conference on Evolutionary Computation Theory and Applications (Рим, Италия, 2014); International Conference on Computational Science (Рейкьявик, Исландия, 2015); The 15th International Multidisciplinary Scientific GeoConference & EXPO SGEM 2015 (Албена, Болгария, 2015); 7th International Conference on Evolutionary Computation Theory and Applications (Лиссабон, Португалия, 2015); Young Science Conference (молодежная конференция, Афины, Греция, 2015); International Conference on Computational Science (Сан-Диего, 2016).

Публикации. По материалам диссертации опубликована 21 печатная работа в изданиях, рекомендуемых ВАК и индексируемых системами Scopus и Web of Science. Получено 4 свидетельства о регистрации программы для ЭВМ.

Личный вклад автора в работах, выполненных в соавторстве, заключается в обосновании требований к оптимизации планирования размещения данных и вычислений; разработке методов планирования с использованием коэволюционного подхода в условиях стационарной и нестационарной среды; разработке алгоритмов оптимизации размещения данных с учетом неоднородности ресурсов хранения по объему и скорости доступа; проработке архитектуры в целом и отдельных модулей, платформ запуска интенсивно-вычислительных задач на больших данных; проведении экспериментальных исследований и интерпретации их результатов.

Структура и объем работы. Диссертационная работа состоит из введения, пяти глав, заключения и списка литературы (181 источник). Содержит 154 страницы текста, включая 44 рисунка и 16 таблиц.

СОДЕРЖАНИЕ РАБОТЫ

Во введении обоснована актуальность исследуемой проблемы, сформулированы цели исследования, отмечена научная новизна и практическая значимость работы.

Первая глава посвящена аналитическому обзору предметной области, включая современные технологии хранения и обработки больших данных, оптимизации перераспределения сложноструктурированных данных и планирования вычислительно-интенсивных задач.

В таблице 1 приведен анализ применимости существующих технологий для задач обработки данных с динамически изменяемой сложной структурой по таким критериям, как возможность планирования реконфигурации ресурсов, на которых осуществляется хранение и обработка; планирование начального размещения данных и их перемещения в процессе работы; возможность работы с различными уровнями обработки и технологиями виртуализации. Данные таблицы демонстрирует преимущества технологии CLAVIRE/DCStorage¹, которая используется в диссертации для организации экспериментальных исследований объекта разработки.

Таблица 1 – Оценка применимости существующих решений для поддержки работы со сложноструктурированными данными

Критерий	MapRack	GlusterFS	LUSTRE	CEPH	Dryad	Drold	Impala (Cloudera)	Drache	Amazon DynamoDB	Clavire DCStorage	Cassandra	MongoDB	Apache Graph / Google Pregel	Mapla	Apache Spark-Streaming	Apache Spark	Apache Storm	MapRack 2.0	MapRack 1.0
Планирование реконфигурации ресурсов	-	-	-	-	+	-	-	-	+	+	-	-	-	-	-	+	+	+	+
Планирование начального размещения данных	+	+	+	+	+	-	-	+	+	+	+	+	+	-	-	+	-	+	+
Планирование перемещения данных в процессе	-	-	-	-	-	-	-	-	-	+	-	-	-	-	-	-	-	-	-
Уровень обработки	1	1	*	*	*	*	*	*	1	*	*	1	*	*	*	*	*	*	*
Виртуализация	-	-	-	-	-	-	-	-	+	+	-	-	-	+	+	+	-	+	-
Многоуровневость	-	-	-	-	-	-	-	-	-	+	-	-	-	-	-	-	-	-	-

Алгоритмы оптимизации перераспределения данных представлены в таблице 2. Видно, что ни один из существующих алгоритмов не покрывает все критерии, это и определяет путь дальнейшего исследования.

¹ Разработана научной школой, к которой принадлежит соискатель.

Таблица 2 – Сравнение существующих алгоритмов перераспределения данных

Критерий	IDP	DDP	DPSGA	RPR	DRAW	ADAPT
Тип реализации алгоритма	GA	GA	GA	GA	GA	GA
История выполнения для <i>пост</i> перераспределения	–	–	–	–	+	–
История выполнения для <i>пред</i> перераспределения	+	+	+	+	–	+
Не увеличивает потребление дискового пространства	+	+	+	–	+	+
Повышает надежность	+/-	+/-	+/-	+/-	+/-	+/-
Повышает производительность системы	+/-	+/-	+/-	+/-	+	+
Поддерживает приоритизацию задач	–	–	–	–	–	–
Поддерживает сетевую оптимизацию	–	–	+	+	–	+
Оптимизация всего плана выполнения MapReduce	–	–	+	+	+	–
Оптимизация «цена решения/скорость работы»	–	+	+	–	–	–
Учет предыдущих решений при поиске новых	–	–	–	–	–	–
Использование уровней хранения	–	–	–	–	–	–

Вторая глава раскрывает особенности реализации вычислительных технологий работы с большими данными. Один из подходов – “код-к-данным” (Code-to-Data) – предполагает выполнение всех расчетов на ресурсах размещения данных, в отличие от традиционного для классических суперкомпьютерных технологий принципа выполнения расчетов на вычислительных узлах (Data-to-Code). Это различие требует рассмотрения гибридной вычислительной архитектуры, эффективно совмещающей обе парадигмы. На рис. 1 приведена такая архитектура, реализованная исходя из логики технологии iPSE (Intelligent Problem Solving Environment), согласно которой функционирует платформа CLAVIRE. Ее особенность – технологическое разделение компонентов, предназначенных для обычных вычислений, и компонентов для вычислительно-интенсивных задач на больших данных.

В работе с целью построения алгоритмов планирования и размещения данных формализованы модели основных вычислительных процессов, реализуемых в рамках этой архитектуры, включая модель распределения данных; модель производительности комплексного приложения; модель реконфигурирования ресурсов.

Модель распределения данных основывается на выражении:

$$T_i = \sum_{j=0}^{Nf_i} \left[\frac{v_{ij}}{Read(s_{ij})} + \frac{v_{ij}}{Conn(s_{ij}, d_i)} + \frac{v_{ij}}{Wrt(d_i)} \right], d_i \neq s_{ij}. \quad (1)$$

С каждой задачей ассоциирован набор файлов, необходимых для ее исполнения $F_i = \{f_j\}, j = \overline{1, Nf_i}$, где Nf_i – число файлов, необходимых для запуска i -й задачи. Объем j -го входного файла i -й задачи обозначим через v_{ij} . $Read(s_{ij})$ – скорость чтения, $Conn(s_{ij}, d_i)$ – скорость передачи данных между узлами s_{ij} и d_i , $Wrt(d_i)$ – скорость записи.

Модель производительности базового приложения (МПП) предполагает выполнение задач в пакетном режиме, она основана на следующем простейшем выражении:

$$T_{exec}(p_m, p_{m_k}, r_u, D_{z_k}) = VM(p_m, r_u) \cdot Perf(p_m, r_u) + U_{i1} T_{sys}^{i1}(p_{m_k}) + U_{i2} T_{inf}^{i2}(p_{m_k}) + U_{i3} T_{data}^{i3}(p_{m_k}, d_z), \quad (2)$$

где P – множество задач в системе; p_m – конкретная задача m , $m = \overline{1, m_{last}}$; $P_{m_k} \subseteq P$; D – множество данных в системе; d_z – конкретные данные z , $z = \overline{1, z_{last}}$; $D_{z_k} \subseteq D$; R – множество ресурсов в системе; r_u – производительность ресурса u , $u = \overline{1, u_{last}}$; $R_{u_k} \subseteq R$.

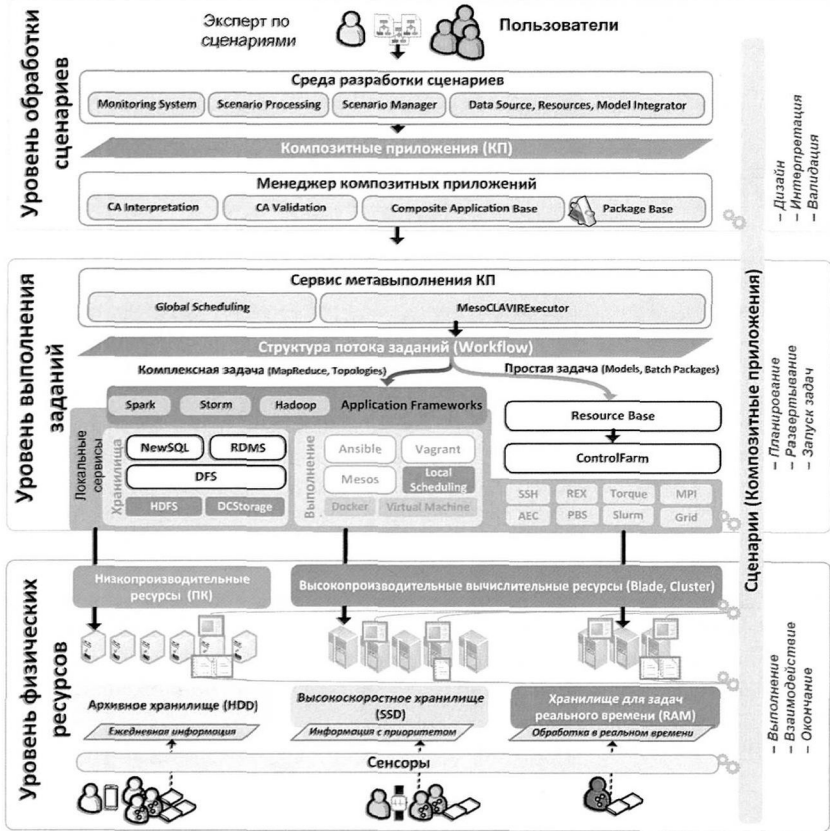


Рисунок 1 – Гибридная вычислительная архитектура, обеспечивающая решение вычислительно-интенсивных задач обработки больших данных

$Perf(p_m, r_u, t)$ определяет модель производительности приложения, полученную благодаря эксперту или только на основе собранной ранее статистики. $VM(p_m, r_u)$ – накладные расходы по инициализации и поддержанию виртуальных ресурсов (виртуальных машин). $U_{i1} T_{sys}^{i1}(p_{m_k})$ – накладные расходы, связанные с различными процессами в системе (программной платформе). $U_{i2} T_{inf}^{i2}(p_{m_k})$ – накладные расходы, связанные с инфраструктурой (каналы связи,

протоколы взаимодействия и т.д.). $\cup_{iz} T_{data}^{iz}(P_{mk})$ – накладные расходы на операции обработки данных (чтение/запись, перемещение, удаление) в различных типах хранилища.

Модель производительности комплексного пакетного приложения (МПКПП). Моделирование процессов предполагает выполнение не только цепочек простых приложений, но и приложений, основанных на своих собственных программных платформах, таких как Apache Hadoop или Apache Spark. Основной особенностью таких приложений является автономное планирование заданного расчета. В этом случае модель производительности составного приложения не может рассматриваться как сумма из набора МПБП, так как появляются дополнительные неучтенные факторы. В этом случае приложение на основе парадигмы *MapReduce* может быть описано выражением:

$$T_{MR}(p_m, P_{mk}, r_u, D_{zk}, N) = \left\lfloor \frac{M}{N} \right\rfloor T_{map}(p_m, P_{mk}, r_u, D_{zk}) + \left\lceil \frac{R}{N} \right\rceil T_{reduce}(p_m, P_{mk}, r_u, D_{zk}) + T_{overhead}, \quad (3)$$

где $T_{map}(p_m, P_{mk}, r_u, D_{zk})$ – фаза Map, состоящая из элементарных операций чтения, сопоставления, сбора и слияния; $T_{reduce}(p_m, P_{mk}, r_u, D_{zk})$ – фаза Reduce, состоящая из элементарных операций перемешивания, слияния, Merge, редуцирования и записи; M – число операций Map; R – число операций Reduce.

Модель реконфигурирования ресурсов. Описывается набором операторов, применяемых на множестве ресурсов. К основным операциям реконфигурации вычислительной среды можно отнести: расширение набора узлов ресурса – *Ext*: $R \rightarrow R$ (новый ресурс образуется как $R' = R \cup r_k$, где r_k – новый узел); сокращение набора узлов ресурса – *Red*: $R \rightarrow R$ ($R' = R/r_k$) и изменение характеристик узла: *Rep*: $R \rightarrow R$ (новый ресурс образуется как $R' = (R/r_i) \cup r'_i$, где r_i – старый узел, r'_i – новый узел); добавление виртуальной машины (ВМ) *Add*: $R \rightarrow R$ ($R' = (R/r_i) \cup r'_i \cup r_j$, где r_i – физический узел, r_j – виртуальный узел; r'_i – измененный физический узел); удаление ВМ: *Rem*: $R \rightarrow R$ ($R' = ((R/r_j)/r_i) \cup r'_i$).

Исходя из моделей, описанных выше, алгоритмы оптимизации размещения больших данных и выполнения операций на них должны:

- используя ретроспективные данные о выполнении задач, настраивать модели МПБП, МПКПП для обеспечения планирования, отражающего текущие условия эксплуатации распределенной среды;
- на основе МПКПП и МПББ генерировать планы реорганизации данных в стационарных и нестационарных условиях между узлами хранения для минимизации накладных расходов (при копировании, ожидании ресурса) обработки больших данных;
- оптимизировать конфигурации виртуальных машин на вычислительных ресурсах для наилучшего распределения вычислительных задач с использованием статистики их выполнения и общей информации;

- совмещать алгоритмы оптимизации распределения данных по узлам и алгоритмы оптимизации конфигураций виртуальных машин в единый механизм многоуровневой оптимизации данных, приложений и ресурсов.

Третья глава содержит постановку задачи оптимизации размещения сложных структур данных в стационарных условиях, создания алгоритмов оптимизации на основе эвристического и метаэвристического подходов, а также экспериментальные исследования разработанных алгоритмов и сравнение их с существующими аналогами.

Функция эффективности размещения данных в базовом случае определяется через число вычислительных узлов, необходимых для обработки задачи T_j : $\varphi(T_j, Pl) = \text{count}\{n_{ij}\}$, где $n_{ij}: \exists < f_i, n_{ij} > \in Pl \ \& \ f_i \in T_j$, n_{ij} – узел хранения данных задачи T_j , а Pl – множество пар файл/узел, отображающих текущее распределение файлов в хранилище. Для каждого уровня n_i определена скорость доступа к данным на этом уровне s_i . Оптимальное распределение представляет собой такое множество Pl_{opt} , что $\forall Pl \ \& \ k_1 + k_2 = 1$, где $0 < k_1 < 1$ и $0 < k_2 < 1$:

$$k_1 \sum_{j=0}^{jMax} \sum_{i=0}^{iMax} \frac{f_{ij}}{s_{ij}} + k_2 \sum_{j=0}^{jMax} c_j \varphi(T_j, Pl_{opt}) \leq k_1 \sum_{j=0}^{jMax} \sum_{i=0}^{iMax} \frac{f_{ij}}{s_{ij}} + k_2 \sum_{j=0}^{jMax} c_j \varphi(T_j, Pl). \quad (4)$$

Коэффициенты k_1 и k_2 отражают вклад составляющих в соотношение: k_1 – вклад скорости доступа к ресурсам, k_2 – вклад оптимального распределения данных; c_j – интегральный показатель важности задачи j ; s_{ij} – скорость доступа к уровню (узлу), на котором находится файл f_{ij} . Для ускорения работы алгоритма в общем случае необходимо оперировать не отдельными файлами, а их наборами, обозначенными как категории, которые отражают принадлежность файлов к группе задач. Набор категорий определен как $C_m^k = \{F_{j_1} \cap F_{j_2} \cap \dots \cap F_{j_k}\} / \{C_m^{j_{k+1}}, \dots, C_m^{jMax}\} \neq \emptyset$, где $m = \overline{1, mMax}$, что согласуется с правилом $\forall f_i \in C_m \ \& \ \forall m' \neq m: f_i \notin C_{m'}$. Для данной постановки реализованы генетический алгоритм (Genetic Algorithm – GA) и алгоритм оптимизации размещения сложных структур данных в стационарных условиях (Categorical Genetic Algorithm, CGA), с модификациями в части категоризации файлов, гравитации, уровней хранения, приоритизации, сетевой составляющей (рисунок 3).



Рисунок 3 – Общая схема работы алгоритма CGA

При реализации алгоритма вводятся понятия составного и уровневого узлов хранения. Составной узел включает себя несколько уровнейых. Данное разделение используется в функции приспособленности, а также при работе алгоритма. Данные распределяются между виртуальными узлами, каждый из которых ассоциируется только с одним уровнем хранения. Алгоритм находит оптимальное размещение данных в многоуровневой системе хранения между всеми виртуальными узлами, учитывая правила репликации в составных узлах хранения. С целью ускорения нахождения решений алгоритмом введено понятие «притяжение файлов», или гравитация:

$$G_{ta} = c_1 G_{own}^n + c_2 G_{near}. \quad (5)$$

Здесь c_1 и c_2 – коэффициенты вкладов двух типов притяжения: собственного уровня хранения, на который выполняется перенос файла, и соседних уровней хранения (в рамках одного составного узла) на оцениваемый.

Значение функции притяжение для собственного уровня хранения для категории файлов находится по следующей формуле:

$$G_{own}^{n_i} = \sum_{c_i: \exists \{t_{c_i}\} \in T_{c_{select}}} \frac{t_l s_{c_i} v_{n_i}}{t_{size}}, \quad (6)$$

где c_i – категория, которая пересекается по задачам с выбранной категорией, для которой происходит расчет c_{select} ; $T_{c_{select}}$ – список задач, к которым принадлежит выбранная категория; s_{c_i} – размер категории на оцениваемом уровне хранения n_i ; v_n – скорость доступа к уровню хранения; t_{size} – размер задачи в используемых файлах; t_l – важность задачи. Для определения силы притяжения соседних уровней выполняется вычисление их собственных значений притяжения с учетом коэффициента влияния уровней на выполнение задачи c_n :

$$G_{near} = \sum_{n_j \in N} c_{n_j} G_{own}^{n_j}, \quad (7)$$

где N – набор уровней хранения, за исключением уже учтенного уровня в собственном притяжении; c_{n_j} – коэффициент влияния конкретного типа уровня хранения на притяжение; n_j – уровень хранения.

Практической реализацией базовой функции $\varphi(T_j, Pl)$ выражения (4) является фитнес-функция следующего вида ($c_t + c_n + c_e = 1$):

$$F_{all} = c_t F_{task} + c_n F_{net} + c_e F_{exec} \quad (8)$$

где F_{all} – эффективность размещения задачи:

$$F_{all} = \sum_i^{tasks} \frac{T_{load}}{(N_{ia} \log_2 N_{ia}) + D_i}, \quad (9)$$

T_{load} – важность i -й задачи, безразмерная величина, отражающая интегральный показатель значимости задачи для вычислительной среды; N_{ia} – минимальное число узлов, необходимое для исполнения задачи i ; D_i – величина накладных расходов при наличии нескольких уровней в иерархии обработки задачи, измеряется в числе уровней. Однако для учета стоимости перехода между текущим размещением файлов и полученным в результате работы алгоритма вводится следующий компонент:

$$F_{net} = 1 - \sum_{j=0}^n \frac{Tr_j * C_j}{Fl_a * C_{max}}, \quad (10)$$

где Fl_a – число всех файлов всех задач; C_{max} – максимальная стоимость пути в

сети; Tr_j – число пересылаемых файлов для узла j ; C_j – стоимость узла j ; n – число всех пересылок между всеми узлами.

Оценить эффективность выполнения задач в хранилище можно в соответствии с формулой:

$$F_{exec} = \sum_{c \in C} \left[\left(1 - \frac{E_c - E_{cmin}}{E_{cmax} - E_{cmin}} \right) \sum_{t \in C} t_{load} \right], \quad (11)$$

где E_c , E_{cmax} , E_{cmin} – текущее, наибольшее и минимальное время выполнения обработки файлов категории.

Результаты работы GA и CGA представлены на рисунке 4 для двух случаев: когда «оригинальные» файлы можно перемещать и нельзя. Алгоритм GA уступает CGA на 17 % в первом случае, и в два раза во втором.

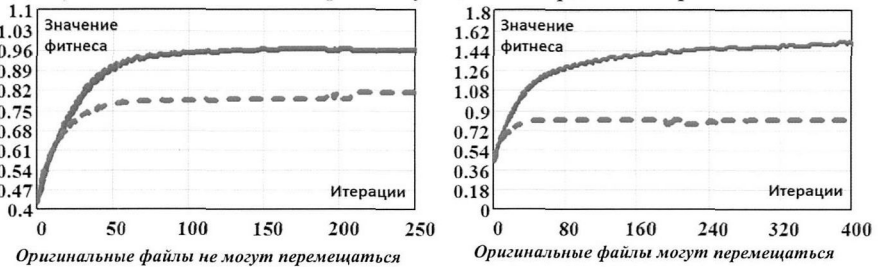


Рисунок 4 – Агрегированные результаты 100 запусков работы CGA (сплошная) и GA (пунктиром) для конфигураций с перемещением «оригинальных» файлов и без

В работе проведены эксперименты по обоснованию выбора коэффициентов фитнес-функции (8), которые показали оптимальное соотношение для c_t , c_n , c_e – 0,7, 0,2, 0,1 соответственно, выполнено сравнение с эвристическими алгоритмами (CRUSH, Greedy²). Эвристические алгоритмы показали результаты хуже, чем GA (до 80 % на определенных сценариях), хотя в целом Greedy предлагает решения лучше CRUSH с приемлемой эффективностью на простых сценариях.

В четвертой главе рассматривается задача оптимизации размещения сложных структур в нестационарных условиях, выполнены разработка и обоснование соответствующего алгоритма Dynamic CGA (DCGA), а также его экспериментальные исследования. В отличие от стационарных условий, в динамической среде статистика выполняемых задач меняется во времени. Это приводит к тому, что эффективность распределения файлов, полученная алгоритмом в момент времени t_i , в большинстве случаев начинает снижаться к t_{i+1} , и в худшем случае к моменту t_{i+k} вместо ускорения возникает замедление расчетов. На рисунке 5 приведены различные состояния окружения, обозначенные в моменты времени t_{i-1} , t_i , t_{i+1} , ..., t_{i+k} , каждый из которых соответствует какому-то событию, которое изменяет состояние системы. В момент времени t_i первоначально созданный план (красная кривая) распределения занимает n итераций до полного завершения. В момент времени

² Классическая реализация жадного алгоритма

t_{i+1} происходит первое событие, которое меняет состояние системы из i в $i + 1$. В этот момент реальная эффективность плана (зеленая кривая) начинает уменьшаться. В момент времени t_{i+2} происходит то же. Из-за возможной высокой частоты изменения среды алгоритм генерирует новую процедуру перераспределения начиная с события k , эффективность которой отмечена на рисунке 5 черным пунктиром.

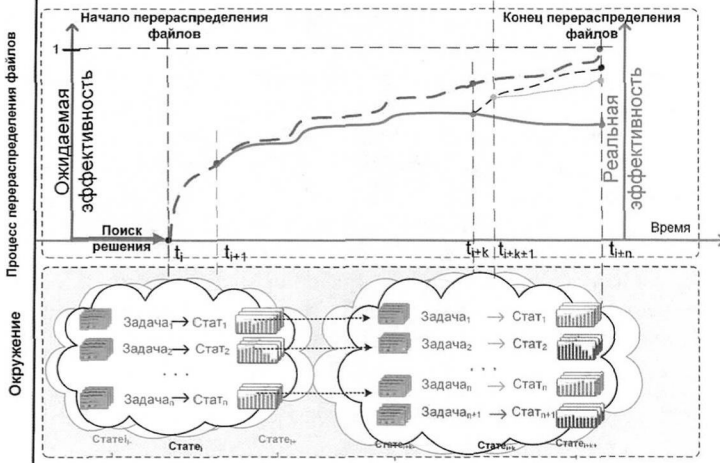


Рисунок 5 – Схема перераспределения данных в динамическом окружении

В этом случае эффективность перераспределения:

$$k_1 \sum_{j=0}^{j_{\max}} c_j \varphi(j, Pl_{opt}) + k_2 \vartheta(Pl, Pl_{opt}) \leq k_1 \sum_{j=0}^{j_{\max}} c_j \varphi(j, Pl') + k_2 \vartheta(Pl', Pl_{opt}), \quad (12)$$

где $\vartheta(Pl, Pl_{opt}) = \sum_{l: n_m^l \neq n_{opt}^l}^{l_{\max}} \rho(f_l, n_m^l, n_{opt}^l)$ – время, необходимое для выполнения плана по перераспределению данных;

$\varphi(j, Pl) = \sum_{l=1}^{l_{\max}} p(f_{lj}, n_m^l, n_{dist}^{lj})$ – время пересылок файлов между узлами;

$(f_{lj}, n_m^l, n_{dist}^{lj}) = \frac{f_l}{ch_{n_m^l n_{dist}^{lj}}}$; $ch_{n_m^l n_{dist}^{lj}}$ – пропускная способность канала передачи

данных между узлами n_m^l и n_{dist}^{lj} . Как следствие, схема на рис. 3 приобретает циклический характер в связи с постоянным обновлением ретроспективных данных и требует учета реализуемых в фоне системы уже выработанных планов.

Для отражения особенностей нестационарных условий фитнес-функция была усовершенствована, а сравнение двух решений происходит по формуле скорости роста фитнеса (СРФ):

$$F_{grf} = \int_{t=0}^{t_{finish}} V(t) \cdot (F_{s1}(t) - F_{s2}(t)) dt, \quad (13)$$

где $V(t)$ – функция памяти системы, отражающая актуальность текущего плана; $F_{s1}(t)$, $F_{s2}(t)$ – фитнес-функция решений $s1, s2$ соответственно. Результаты сравнения эффективности алгоритмов для динамической среды: случайное распределение – 0,27, CRUSH – 0,43, Greedy – 0,51, CGA – 0,96. Как видно,

результаты алгоритма без учета динамической составляющей на 88 % лучше результатов Greedy. Таким образом, даже в динамическом окружении статический CGA остается эффективней «жадного» алгоритма.

Результаты, полученные для DCGA и фитнес-функции с использованием СРФ, для всех конфигураций вычислительной среды показывают устойчивое поведение – каждое последующее решение лучше предыдущего. При этом использование СРФ наиболее эффективно при сильной изменчивости внешних условий. Так, например, для ситуации с частотой изменения 4, значимости (силой) модификации 6 и сетевым коэффициентом 0,5 наблюдается прирост эффективности на всех этапах изменения среды (4,6; 6,4; 7,1; 8%), в то время как DCGA без СРФ показывает (0; 0,3; 0,6; 0,9%), а при сетевом коэффициенте 0,3 ведет себя нестабильно. При этом DCGA с СРФ показывает максимальный прирост производительности в общем (до 20%).

В пятой главе рассмотрены вопросы организации коэволюционных алгоритмов планирования вычислительно-интенсивных задач на больших данных. Принимая во внимание все критерии оптимизации и ограничения, можно определить формальную постановку задачи. Цель задачи планирования – найти такое расписание задач на ресурсы S_{opt} , чтобы $\forall S' \neq S_{opt}$:

$$\sum_i^{|G|} \alpha_i g_i(S_{opt}) - \sum_j^{|C|} \beta_j c_j(S_{opt}) \geq \sum_i^{|G|} \alpha_i g_i(S') - \sum_j^{|C|} \beta_j c_j(S'). \quad (14)$$

где сумма при α_i – критерии оптимизации; при β_j – выполняемые ограничения.

Для решения данной задачи разработан Dynamic Coevolution Genetic Algorithm (DCoGA) для нестационарной среды, позволяющий в начале фазы планирования или перепланирования расчетов изменять конфигурацию ресурсов, DCoGA в свою очередь, является основой для нового алгоритма планирования с учетом динамической реконфигурации ресурсов во времени Dynamic Resources CoGA(DRCoGA), см. рис. 6.

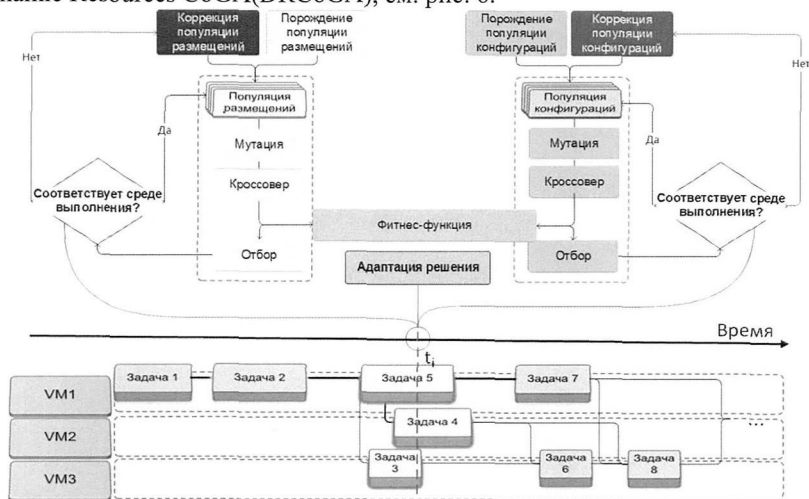


Рисунок 6 – Схема выполнения DCoGA в динамической среде

В DRCoGA параллельно происходят планирование задач и планирование ресурсов, которые постоянно адаптируются к изменяющимся условиям вычислительной среды. При этом генерируются два плана выполнения во времени и для задач, и для вычислительных ресурсов (традиционно план генерируется только для задач) – тем самым система максимально подстраивается под оптимизацию выполнения текущих расчетов.

Алгоритм Data Aware DRCoGA (DACoGA) позволил совместить в себе принципы коэволюции DRCoGA с алгоритмами перераспределения данных DCGA. Результаты эффективности работы алгоритмов по сравнению с традиционным для таких задач алгоритмом HEFT представлены в таблице 3.

Таблица 3 – Прирост эффективности алгоритмов по отношению к HEFT (%)

Композитное приложение	Алгоритм		
	GA	DCoGA	DACoGA
Montage 25	36,4	51,0	51,4
Montage 50	27,6	28,6	33,3
CyberShake 30	3,1	51,5	51,5
CyberShake 50	10,9	13,6	32,8
Inspiral 30	0,2	4,9	15,6
Inspiral 50	5,3	9,5	17,0

Из таблицы следует, что DACoGA является максимально эффективной схемой и демонстрирует от 15 до 52% прироста в скорости выполнения задач. На рисунке 7 представлена зависимость эффективности решения от числа выполняемых итераций алгоритма (число популяций). DACoGA в большинстве случаев требует больше итераций для сходимости (1000–1500 против 500–1000 CoGA) из-за более сложной реализации.

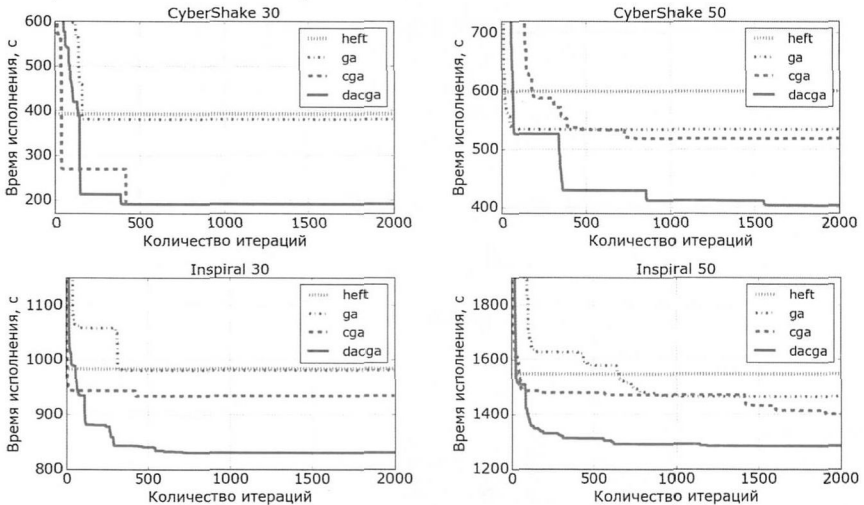


Рисунок 7 – Зависимость времени выполнения расчетов от числа итераций алгоритмов

Заключение. В ходе выполнения диссертационной работы получены следующие результаты:

- на основе аналитического обзора сформулированы основные требования к математическому и программному обеспечению для планирования и распределения сложноструктурированных больших данных в вычислительно-интенсивных приложениях;
- разработаны математические модели, в параметрической форме описывающие распределенную среду, процессы выполнения композитных приложений и перераспределения данных, что позволило сформулировать общую постановку оптимизационных задач;
- разработаны и экспериментально исследованы алгоритмы CRUSH (модификация), Greedy, разновидности GA и CGA оптимизации перераспределения сложноструктурированных данных для статической и вычислительной среды с учетом параметров выполняемых задач, собранных по ретроспективе вычислительной нагрузки системы;
- разработаны и экспериментально исследованы алгоритмы Greedy в динамике, DCGA с модификациями (СРФ), которые оптимизируют перераспределения сложноструктурированных данных для динамической вычислительной среды с учетом параметров выполняемых задач, собранных по ретроспективе вычислительной нагрузки системы;
- разработано и исследовано семейство алгоритмов динамического коэволюционного планирования композитных приложений (DCoGA, DRCoGA, DACoGA) на динамически реконфигурируемую вычислительную инфраструктуру с учетом особенностей обработки больших данных.

Список публикаций по теме диссертации

1. Чуров Т. Н., Насонов, Д. А., Штейнберг, Б. Я., Бухановский, А. В. Особенности вероятностного анализа производительности и надежности проблемно-ориентированных сред облачных вычислений //Известия высших учебных заведений. Приборостроение. – 2011. – Т. 54. – №. 10. С. 51-57. Объем: 0.5 п.л. (авторский вклад 0.18 п.л.) **(соответствует перечню ВАК)**
2. Мостаманди С. В., Насонов Д. А., Калюжная А., Бухановский А. В. Ансамблевые прогнозы экстремальных гидрометеорологических явлений в распределенной среде CLAVIRE //Известия высших учебных заведений. Приборостроение. – 2011. – Т. 54. – №. 10. Объем: 0.38 п.л. (авторский вклад 0.19 п.л.) **(соответствует перечню ВАК)**
3. Knyazkov K. V., Nasonov D. A., Tchurov T. N., Boukhanovsky A. V. Interactive workflow-based infrastructure for urgent computing. // Procedia Computer Science. – 2013. Vol. 18 – P. 2223-2232. 0.63 п.л. (авторский вклад 0.21 п.л.) **(в Scopus, соответствует перечню ВАК)**
4. Nasonov D., Butakov N. Hybrid Scheduling Algorithm in Early Warning Systems //Procedia Computer Science. – 2014. – Vol. 29. – P. 1677-1687. Объем: 0.75 п.л. (авторский вклад 0.4 п.л.) **(в Scopus, соответствует перечню ВАК)**
5. Kosukhin S. S., Kalyuzhnaya A. V., Nasonov D. Problem Solving Environment for

- Development and Maintenance of St. Petersburg's Flood Warning System //Procedia Computer Science. – 2014. – Vol. 29. – P. 1667-1676. Объем: 0.63 п.л. (авторский вклад 0.12 п.л.) **(в Scopus, соответствует перечню ВАК)**
6. Razumovskiy, A., Spivak, A., Nasonov, D., & Boukhanovsky, A.V. Evolutionary data reorganization for efficient workload processing //AICT, 2014 IEEE 8th International Conference on. – IEEE. 2014. – P. 1-6. Объем: 0.5 п.л. (авторский вклад 0.16 п.л.) **(в Scopus, соответствует перечню ВАК)**
 7. Nasonov D.A., Churov T.N., Zagarskikh A.S., Kovalchuk S.V. Technological platform for complex processing of large data within early warning systems //WIT Transactions on Information and Communication Technologies. – 2014. Vol. 56. – P. 625-634. Объем: 0.63 п.л. (авторский вклад 0.39 п.л.) **(в Scopus, соответствует перечню ВАК)**
 8. Nasonov D., Butakov N., Balakhontseva M., Knyazkov K., Boukhanovsky A. V. Hybrid Evolutionary Workflow Scheduling Algorithm for Dynamic Heterogeneous Distributed Computational Environment //International Joint Conference SOCO'14. – Springer International Publishing, 2014. – P. 83-92. Объем: 0.63 п.л. (авторский вклад 0.5 п.л.) **(в Scopus, соответствует перечню ВАК)**
 9. Butakov N.A., Nasonov D., Boukhanovsky A.V. Evolutionary Inheritance in Workflow Scheduling Algorithms within Dynamically Changing Heterogeneous Environments // ECTA 2014 - Proceedings of the International Conference on Evolutionary Computation Theory and Applications, 2014. – P. 160-168. Объем: 0.65 п.л. (авторский вклад 0.16 п.л.) **(в Scopus, соответствует перечню ВАК)**
 10. Butakov N., Nasonov D. Co-evolutional genetic algorithm for workflow scheduling in heterogeneous distributed environment // 8th IEEE International Conference on Application of Information and Communication Technologies, AICT 2014 - Conference Proceedings, 2014. – P. 7035936 Объем: 0.5 п.л. (авторский вклад 0.2 п.л.) **(в Scopus, соответствует перечню ВАК)**
 11. Visheratin A.A., Nasonov D.A., Kaluzhnaya, A.V., Kosukhin S.S. A simulation platform for atmospheric phenomena study within coastal floods in Baltic sea area // GeoConference-SGEM: 15th International Multidisciplinary Scientific Geoconference SGEM 2015. 2015. Vol. 1. N 2. – P. 11-18. Объем: 0.5 п.л. (авторский вклад 0.15 п.л.) **(в Scopus, соответствует перечню ВАК)**
 12. Nasonov D.A., Visheratin A.A., Knyazkov K.V., Kovalchuk S.V. Interactive e-Science Cyberinfrastructure for Workflow Management Coupled with Big Data Technology // GeoConference-SGEM: 15th International Multidisciplinary Scientific Geoconference SGEM 2015. 2015. Vol. 1. N 2. – P. 175-182. Объем: 0.5 п.л. (авторский вклад 0.15 п.л.) **(в Scopus, соответствует перечню ВАК)**
 13. Zagarskikh A.S., Karsakov A., Mukhina K.D., Nasonov D.A. An Efficient Approach of Infrastructure Processing Visualization within Cloud Computing Platform //Procedia Computer Science. – 2015. – Vol. 66. – P. 705-710. Объем: 0.5 п.л. (авторский вклад 0.08 п.л.) **(в Scopus, соответствует перечню ВАК)**
 14. Visheratin A.A., Melnik M., Butakov N., Nasonov D.A. Hard-deadline constrained workflows scheduling using metaheuristic algorithms //Procedia Computer Science. – 2015. – Vol. 66. – P. 506-514. Объем: 0.63 п.л. (авторский вклад 0.14 п.л.) **(в Scopus, соответствует перечню ВАК)**

15. Nasonov D.A., Melnik M., Shindyapina N., Butakov N. Metaheuristic coevolution workflow scheduling in cloud environment // IJCCI, 2015 7th International Joint Conference on. – SCITEPRESS, 2015. – Vol. 1. – P. 252-260. Объем: 0.63 п.л. (авторский вклад 0.09 п.л.) **(в Scopus, соответствует перечню ВАК)**
16. Bochenina K.O., Butakov N., Dukhanov A.V., Nasonov D. A clustering-based approach to static scheduling of multiple workflows with soft deadlines in heterogeneous distributed systems // Procedia Computer Science. – 2015. – Vol. 51. – P. 2827-2831. Объем: 0.5 п.л. (авторский вклад 0.08 п.л.) **(в Scopus, соответствует перечню ВАК)**
17. Spivak A.I., Razumovskiy A.V., Miagkov A.V., Nasonov D.A. Evolutionary Replicative Data Reorganization with Prioritization for Efficient Workload Processing // Procedia Computer Science. – 2015. – Vol. 51. – P. 2357-2366. Объем: 0.63 п.л. (авторский вклад 0.23 п.л.) **(в Scopus, соответствует перечню ВАК)**
18. Kalyuzhnaya A.V., Visheratin A.A., Dudko A., Nasonov D.A., Boukhanovsky A.V. Synthetic storms reconstruction for coastal floods risks assessment // Journal of Computational Science. – 2015. – Vol. 9. – P. 112-117. Объем: 0.5 п.л. / 0.18 п.л. **(в Scopus, соответствует перечню ВАК)**
19. Visheratin A. A., Melnik M. A., Nasonov D. Workflow Scheduling Algorithms for Hard-deadline Constrained Cloud Environments // Procedia Computer Science. – 2016. – Vol. 80. – P. 2098-2106. Объем: 0.75 п.л. (авторский вклад 0.49 п.л.) **(в Scopus, соответствует перечню ВАК)**
20. Butakov N., Nasonov D., Svitenkov A., Radice A., Boukhanovsky A.V. Quality-based Approach to Urgent Workflows Scheduling // Procedia Computer Science. – 2016. – Vol. 80. – P. 2074-2085. Объем: 0.75 п.л. (авторский вклад 0.24 п.л.) **(в Scopus, соответствует перечню ВАК)**
21. Visheratin A.A., Trofimenko T.B., Mukhina K.D., Nasonov D.A., Boukhanovsky A.V. Urgent Information Spreading Multi-layer Model for Simulation in Mobile Networks // Procedia Computer Science. – 2016. – Vol. 80. – P. 2086-2097. Объем: 0.88 п.л. (авторский вклад 0.22 п.л.) **(в Scopus, соответствует перечню ВАК)**
22. Распределенная программная система хранения данных вычислительных экспериментов DStorage / К.В. Князьков, А.В. Разумовский, А.И. Спивак, Т.Н. Чуров, Д. Насонов. Св-во о гос. регистрации программы для ЭВМ №2012618271 от 2 октября 2012 г.
23. Ядро многопрофильной инструментально-технологической платформы облачных вычислений CLAVIRE / К.В. Князьков, С.В. Марьин, Т.Н. Чуров, А.В. Ларченко, Д. Насонов, А.В. Бухановский. Св-во о гос. регистрации программы для ЭВМ №2012618267 от 2 октября 2012 г.
24. Программный комплекс вычислительного эксперимента в области морских гидрологических прогнозов / С.В. Иванов, С.С. Косухин, Д.А. Насонов. Св-во о гос. регистрации программы для ЭВМ № 2012618533 от 8.10.2012 г
25. Программа моделирования процесса исполнения композитных приложений в гетерогенных динамических средах / Н.А. Бутаков, Д. Насонов. Св-во о гос. регистрации программы для ЭВМ № 2015614298 от 14.04.2015 г

Формат: 60x84 1/16. Печать цифровая. Бумага офсетная.
Гарнитура Times. Заказ: 479. Тираж: 100экз.
Учреждение «Университетские телекоммуникации»
Санкт-Петербург, Саблинская ул., д.14, Биржевая линия, д.16
+7(812)9151454, zakaz@tibir.ru, www.tibir.ru